RUL Prediction With C-MAPSS Dataset

Xuning Zhang
Student BTH
Karlskrona, Sweden
zhxu24@student.bth.se

I. Introduction

The fundamental concept of prognostics and health management (PHM) is to find an approach to evaluate the system's health and predict its remaining useful life (RUL). In the era of digital transformation, many methods and algorithms from the data science world have been applied to use for PHM modeling. One of the leading algorithms is machine learning. [1]

There are a lot of running project working on RUL prediction of turbofan engine. This project focuses on predictive maintenance of turbofan engines using the C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) dataset.The goal is to predict the RUL of engines, which is critical for reducing maintenance costs and preventing unexpected failures. The task is framed as a time-series regression problem, where historical sensor data is used to estimate the remaining operational time before failure.

The primary data mining problem addressed is predictive modeling in a high-dimensional, time-series dataset. This involves preprocessing the data, selecting relevant features, and developing a robust machine learning model. The solution leverages Long Short-Term Memory (LSTM) networks, a type of recurrent neural network (RNN) known for its effectiveness in handling sequential data.

II. Methodology

To have a more comprehensive understanding and implementation of the project, a very useful methodology - CRISP-DM (Cross-Industry Standard Process for Data Mining) was token here which consists of six phases:1) business understanding, 2) data understanding, 3) data preparation, 4) modeling, 5) evaluation, and 6) deployment. [2]

A. Business Understanding

Predictive maintenance is crucial in industries like aviation, where engine failures can lead to catastrophic consequences. The goal is to minimize downtime and maintenance costs by accurately predicting RUL.

There has been plenty of relative researches or works on Kaggle and Github. They introduce the basic understanding of C-MAPSS dataset and different machine learning approaches for this problem. Most of the works are based on deep learning models because of the complexity of this dataset and timeline. Hybrid deep learning model using CNN-LSTM using RMSE as evaluation is the most common solution and getting the results about 15-20 RMSE. Further discussions are published on Google Scholar.

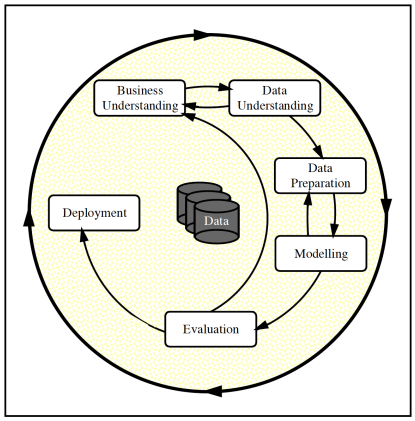

Fig. 1. Phases of the CRISP-DM reference model [3]

Based on all these relative works. Deep learning model with time-series like RNN and LSTM were considered in this project. With comparison of these two, LSTM is for processing long sequence tasks and can capture long-term dependencies which is suitable for C-MAPSS dataset. So LSTM was finally determined to be the main method in this predicting project.

B. Data understanding

For C-MAPSS dataset, it is consist of four different subsets from FD001 to FD004. The difference between these subsets is whether it is single-case or multi-case when sensors are working. In this project, FD001 would be used as the main dataset because single-case working has obvious monotonic change characteristics during the operation cycle which is easier for machine learning model to predict.

Information of the C-MAPSS dataset.

	C-MAPSS			
Dataset	FD001	FD002	FD003	FD004
Engine units for training	100	260	100	249
Engine units for testing	100	259	100	248
Operating conditions	1	6	1	6
Fault modes	1	1	2	2
Training samples (default)	17,731	48,819	21,820	57522
Testing samples	100	259	100	248

The dataset contains sensor data from multiple turbofan engines over time. Each engine starts with different degrees of initial wear and manufacturing variation, and it operates

normally until a failure occurs. It includes 21 sensor readings, operational settings, and a timestamp with RUL as its target variable.Each engine has a varying number of operational cycles.

The dataset contains no missing values, but some sensors are constant or redundant.

C. Data preparation

Data preprocessing included following steps in this project:

1) Data Cleaning and Visualization: First,using unit_number and time_cyclessetting_1, setting_2, setting_3 and sensor_1 to sensor_21 to decorate the original dataset. Dropping redundant columns: Columns 26 and 27 were removed as they contained no useful information. The RUL for each engine was computed as the difference between the maximum time cycle and the current cycle.

Second, different plotting methods were implemented to visualize some parts of the data which showing a more intuitive vision and understanding.

What's more,sensor data was standardized using StandardScaler to ensure consistent scaling.

	count	mean	std	min	25%	50%	75%	max
unit_number	20631.0	51.506568	2.922763e+01	1.0000	26.0000	52.0000	77.0000	100.0000
time_cycles	20631.0	108.807862	6.888099e+01	1.0000	52.0000	104.0000	156.0000	362.0000
setting_1	20631.0	-0.000009	2.187313e-03	-0.0087	-0.0015	0.0000	0.0015	0.0087
setting_2	20631.0	0.000002	2.930621e-04	-0.0006	-0.0002	0.0000	0.0003	0.0006
setting_3	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100.0000
sensor_1	20631.0	518.670000	0.000000e+00	518.6700	518.6700	518.6700	518.6700	518.6700
sensor_2	20631.0	642.680934	5.000533e-01	641.2100	642.3250	642.6400	643.0000	644.5300
sensor_3	20631.0	1590.523119	6.131150e+00	1571.0400	1586.2600	1590.1000	1594.3800	1616.9100
sensor_4	20631.0	1408.933782	9.000605e+00	1382.2500	1402.3600	1408.0400	1414.5550	1441.4900
sensor_5	20631.0	14.620000	1.776400e-15	14.6200	14.6200	14.6200	14.6200	14.6200
sensor_6	20631.0	21.609803	1.388985e-03	21.6000	21.6100	21.6100	21.6100	21.6100
sensor_7	20631.0	553.367711	8.850923e-01	549.8500	552.8100	553.4400	554.0100	556.0600
sensor_8	20631.0	2388.096652	7.098548e-02	2387.9000	2388.0500	2388.0900	2388.1400	2388.5600
sensor_9	20631.0	9065.242941	2.208288e+01	9021.7300	9053.1000	9060.6600	9069.4200	9244.5900
sensor_10	20631.0	1.300000	0.000000e+00	1.3000	1.3000	1.3000	1.3000	1.3000
sensor_11	20631.0	47.541168	2.670874e-01	46.8500	47.3500	47.5100	47.7000	48.5300
sensor_12	20631.0	521.413470	7.375534e-01	518.6900	520.9600	521.4800	521.9500	523.3800
sensor_13	20631.0	2388.096152	7.191892e-02	2387.8800	2388.0400	2388.0900	2388.1400	2388.5600
sensor_14	20631.0	8143.752722	1.907618e+01	8099.9400	8133.2450	8140.5400	8148.3100	8293.7200
sensor_15	20631.0	8.442146	3.750504e-02	8.3249	8.4149	8.4389	8.4656	8.5848
sensor_16	20631.0	0.030000	1.387812e-17	0.0300	0.0300	0.0300	0.0300	0.0300
sensor_17	20631.0	393.210654	1.548763e+00	388.0000	392.0000	393.0000	394.0000	400.0000
sensor_18	20631.0	2388.000000	0.000000e+00	2388.0000	2388.0000	2388.0000	2388.0000	2388.0000
sensor_19	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100.0000
sensor_20	20631.0	38.816271	1.807464e-01	38.1400	38.7000	38.8300	38.9500	39.4300
sensor_21	20631.0	23.289705	1.082509e-01	22.8942	23.2218	23.2979	23.3668	23.6184

Fig. 2. data review

2) Feature Engineering: By visualizing and analyzing different features, some constant features which were redundant to the dataset were found:'setting_3', 'sensor_1', 'sensor_5', 'sensor_10', 'sensor_16', 'sensor_18', 'sensor_19'. They were removed from the dataset to reduce noise.

3) Processing Inputs And Targets: Time window were needed according to the characteristics of the chosen model.So different functions were implemented to handle target value(RUL) and windowing data with a specified window

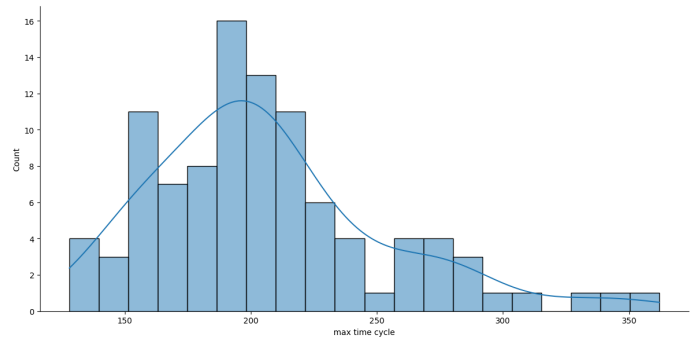

Fig. 3. time-cycle conditions

length and shift size. Also, using label smoothing on targets to reduce the model's sensitivity to extreme values, thereby improving generalization ability.

D. Modeling

The LSTM model was chosen due to its ability to capture temporal dependencies in sequential data.LSTM was introduced in 1997 by Hochreiter and Schmidhuber [4]. Instead of containing all data in long time-steps, LSTM adopts the gating structure to control the flow of the information. The corresponding structure diagram of the LSTM memory cell is seen in Figure 4. The memory cell of LSTM consists of the forget gate, input gate, and output gate. The LSTM

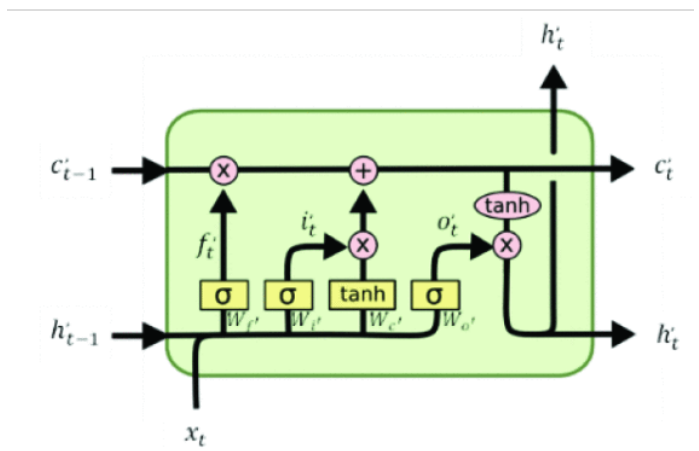

Fig. 4. LSTM memory cell [4]

layers take the input from the reshaped-input data using sliding time window processing. The features were contained in the predefined time windows of size, T_w. However, the input dimension was considered as an imaginary time dimension. This allows the LSTM layers to separate the time dimension from the input data for the recurrent training process. In the LSTM layer, a set of memory cells consists of three non-linear gating units (Figure 4 illustrates a unit of LSTM memory cell) that update a cell state, $C_t \in \mathbb{R}^{h'}$ in each iteration, using a hidden state vector, $h'_{t-1} \in \mathbb{R}^{h'}$, with input $x_{I_t} \in \mathbb{R}^q$, where h

is the recurrent time dimension of the LSTM layer. Operation formulas of the LSTM layers are defined as:

$$f'_t = \sigma\left(W_{f'}x^i_t + R_{f'}h_{t-1} + b_{f'}\right) \tag{1}$$

$$i'_t = \sigma\left(W_{i'}x^i_t + R_{i'}h_{t-1} + b_{i'}\right) \tag{2}$$

$$o'_t = \sigma\left(W_{o'}x^i_t + R_{o'}h_{t-1} + b_{i'}\right) \tag{3}$$

where f'_t, i'_t, and o'_t are forget gate, input gate, and output gate respectively, σ is a sigmoid activation function inside the LSTM cell used to scale the output to the 0-1 range, $W \in \mathbb{R}^{h\times q}$ are input weight matrices, $R \in \mathbb{R}^{h\times h}$ are the recurrent weight matrices, and $b \in \mathbb{R}^h$ are bias vectors. All f'_t, i'_t, and o'_t are first calculated and fed through the gate to compute the new cell state $\tilde{C}_t$ using hyperbolic tangent (tanh) activation function as follows:

$$\tilde{C}_t = \tanh\left(W_C x^i_t + R_C h_{t-1} + b_C\right) \tag{4}$$

Then, the previous cell state C_{t-1} were updated into the new cell state Ct as seen in Figure 4, or as the formula:

$$\tilde{C}_t = f'_t \otimes C_{t-1} + i'_t \otimes \tilde{C}_t \tag{5}$$

Finally the new hidden state, h_t can be calculated by employing hyperbolic tangent activation function to the current cell state times the result from the output gate:

$$h_t = o_t \otimes \tanh\left(C_t\right) \tag{6}$$

Our model architecture includes:Three LSTM layers with decreasing hidden units (128, 64, 32). Fully connected layers for regression output.

Hyperparameters:

1) Window length: 30

2) Early RUL: 125

3) Learning rate: 0.001 (with a scheduler to reduce it after 5 epochs)

E. Evaluation

To measure and evaluate the performance of the deep learning model, root mean square error (RMSE) was used. RMSE is commonly used as a performance indicator for regression models especially in aircraft area. The following is the formula of RMSE:

$$RMSE = \sqrt{\frac{1}{n}\sum\nolimits_{i=1}^{n}\left[y_i - \bar{y}_i\right]^2} \tag{7}$$

Also, S-score was also used as a custom metric penalizing late predictions more than early ones:

$$S_{\text{score}} = \sum\left(\begin{cases} e^{-\frac{\Delta}{13}} - 1 & \text{if } \Delta < 0 \\ e^{\frac{\Delta}{10}} - 1 & \text{if } \Delta \geq 0 \end{cases}\right)$$

where $\Delta = \text{rul_pred} - \text{rul_true}$.

F. Deployment

To find the best model and best parameter combination, the LSTM model was deployed with different epoch numbers such as 10,20,40,80,160 in a production environment. Then plotting the real-time training and validation loss graph to moniter model performance over time. Finally,create this final report with visualizations, model performance, and recommendations to analyze this project.

III. Results and Analysis

A. Model Performance

For this LSTM model, different epoch numbers were implemented with Adaptive Moment Estimation (Adam) as the optimizer. In this case, the more epochs you got, the higher the RMSE value seems to be. Also S-score was also affected by the parameter to some extent.Detailed results were shown in the following table:

epoch	RMSE	S-score
10	14.0468	298.5
20	15.4826	383.8
40	15.5719	435.9
80	15.9154	483.4
160	19.8698	762.4

TABLE I
RESULTS

B. Visualization

Plots of true vs. predicted RUL showed a close alignment, though some deviations are observed for engines with shorter lifespans.

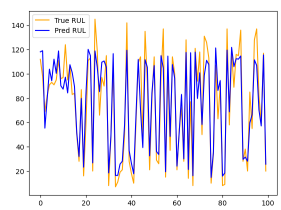

Fig. 5. epoch = 10

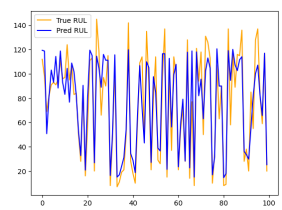

Fig. 6. epoch = 20

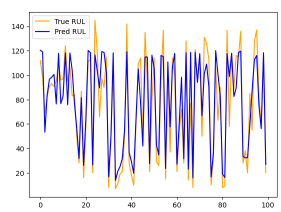

Fig. 7. epoch = 80

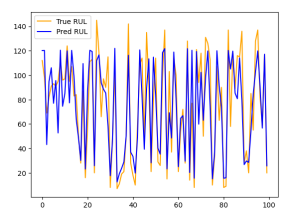

Fig. 8. epoch = 160

Fig. 9. Plots of true vs. predicted RUL

IV. Discussion

A. Results

By comparing these results, the relatively best performance was RMSE=15.9154,S-score=483.4 with epoch=80.HJgher S-score is generally considered better because it means that the predicted remaining useful life (RUL) is closer to the actual RUL, also having a decent RMSE with lower error and higher performance. However, too high a number of epoch would increase the risk of model overfitting causing a unstable RMSE value even though S-score is nice.

B. Strengths

1) Temporal Pattern Capture: The LSTM model excels at capturing temporal dependencies in the sensor data, which is critical for predicting RUL in time-series datasets. Its ability to remember long-term patterns ensures accurate predictions even for engines with complex degradation trends.

2) Feature Selection: By removing redundant and constant sensors, the model focuses on the most informative features, reducing noise and improving performance. Evaluation Metrics: The use of both RMSE and the S-score provides a balanced evaluation. The S-score, in particular, penalizes late predictions more heavily, aligning with the goal of avoiding unexpected failures.

3) Scalability: The preprocessing and modeling pipeline can be adapted to similar predictive maintenance tasks in other industries, such as manufacturing or energy.

C. Comparison to Previous Work

The approach aligns with existing literature on using LSTMs for RUL prediction. However, the inclusion of the S-score provides a more nuanced evaluation metric. Also, the use of a learning rate scheduler and early RUL adjustment enhances model performance as Hyperparameter Tuning.

D. Explainable and Interpretable

To integrate XAI knowledge to this project,lots of efforts were made to improve the interpretability of the model:

1) Feature Importance: By removing irrelevant sensors, the model focuses on the most critical features, making it easier to understand the driving factors behind predictions.

2) Visualization: Using different packages to describe and display multiple details and structures of FD001 data.Plots of true vs. predicted RUL provide insights into the model's performance and highlight areas for improvement.

V. Limitations

A. Data-Related

1) Simulated Data: The C-MAPSS dataset is generated using simulations rather than real-world data. While it is a valuable benchmark, it may not fully capture the complexities of actual engine behavior.

2) Limited Operating Conditions: The dataset assumes a fixed set of operating conditions, which may not reflect the variability encountered in real-world scenarios.

B. Model-Related

1) Overfitting Risk: Despite putting some efforts to reduce overfitting, the model may still struggle with generalization due to the small dataset size. As the training epoch increasing, the model would be more like to be overfitting.

2) Black-Box Nature: LSTMs are inherently difficult to interpret, which could be a barrier to adoption in industries where explainability is critical.

C. Future Work

1) Model Interpretability: Techniques like SHAP or LIME could be used to enhance the interpretability of the model and provide actionable insights.

2) Hybrid Model: Using a hybrid model like CNN-LSTM may improve various aspects of the algorithm's performance and reduce the risk of overfitting.

VI. Conclusions

In summary the experiments and models introduced in this work are reproductions and extensions from the previous published article. In this work, both feature selection and relative modeling were performed.

The reported results were obtained from training and testing the model five times. RMSE and S-score were considered and compared to get the final results. The results from the five times training and testing solved the goal of less than 20 RMSE proposed in the project proposal.

The project demonstrates the effectiveness of LSTM networks for predicting RUL in turbofan engines. Key takeaways include the importance of preprocessing and the need for robust evaluation metrics.Future work could explore incorporating additional data sources and improving model interpretability.

References

[1] P. Khumprom, A. Davila-Frias, D. Grewell and D. Buakum, "A Hybrid Evolutionary CNN-LSTM Model for Prognostics of C-MAPSS Aircraft Dataset," 2023 Annual Reliability and Maintainability Symposium (RAMS), Orlando, FL, USA, 2023, pp. 1-8

[2] Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T.P., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide.

[3] R. Wirth and J. Hipp, 2000, In Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining (Vol. 1, pp. 29-39).

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory", Neural computation, vol. 9, no. 8, pp. 1735-1780, 1997.